D

TAREQO

19 — Migration de la clôture de caisse quotidienne sur le Supabase de production (runbook)

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Étape 0 — Prérequis 4

Étape 1 — Sauvegarde (obligatoire) 4

Étape 2 — Pré-vol (lecture seule, SQL Editor) 5

Étape 3 — Appliquer `supabase/upgrade.sql` 5

- Option A — psql via le pooler, UNE transaction (recommandé — convention du projet) 5
- Option B — SQL Editor du Dashboard 6

Étape 4 — Vérification post-migration (lecture seule) 6

Étape 5 — Idempotence (optionnelle, confirme que la réapplication ne duplique pas) 6

Étape 6 — Publier l'app (rapport Z) 7

Rollback (seulement si nécessaire) 7

Checklist finale 7

Objectif : appliquer la Partie 25 de supabase/upgrade.sql au projet de production (dotklbpqmlnhcdifiukd) — active la **clôture de caisse quotidienne (rapport Z)** sur la base EXISTANTE : la table cash_closings (une clôture par magasin par jour) + le RPC close_cash_register qui calcule le montant attendu du tiroir-caisse CÔTÉ SERVEUR et persiste la clôture atomiquement.

Ce que fait la Partie 25 (idempotente — mêmes conventions que les Parties 7/8/22) :
1. 25.1 — table cash_closings (instantané du jour : total des ventes, tiroir attendu, caisse comptée, ventilation par méthode de paiement en JSONB) avec un UNIQUE `(store_id, closing_date)` — re-clôturer le même jour fait un UPDATE, jamais un doublon ;
2. 25.2 — RLS + politiques : lecture pour le propriétaire/les employés du magasin ; écritures via le RPC (INSERT pour son propre magasin ; UPDATE/DELETE uniquement sur ses propres lignes — défense en profondeur) ;
3. 25.3 — index idx_cash_closings_store (analyses scopées magasin) ;
4. 25.4 — triggers user_id/store_id (mêmes aides que le schéma) ;
5. 25.5 — RPC close_cash_register(text, timestamptz, bigint), SECURITY DEFINER avec la garde `is_manager_of_store` (la clôture est une opération MANAGÉRIALE — propriétaire/gérant uniquement), calcul du caisse attendue côté serveur (Σ des ventes caisse ACTIVES du jour — carte/M-Pesa/E-Mola/transfert ne passent pas par le tiroir physique) et un UPSERT en UNE transaction.
> La même application amène aussi la Partie 26 : le RPC en lecture seule get_cash_closing_summary(text, timestamptz) — l'APERÇU du rapport Z agrégé CÔTÉ SERVEUR (mêmes critères que close_cash_register, gardé par is_my_store), pour que les magasins avec de gros historiques ne chargent plus les ventes sur le client (échelle H2, modèle get_cash_flow/get_sales_report).

Le script est 100% idempotent et la Partie 25 suit les mêmes conventions

> que les Parties 7/8/22 (CREATE TABLE IF NOT EXISTS, DROP POLICY IF EXISTS + CREATE POLICY, DROP TRIGGER IF EXISTS + CREATE TRIGGER, CREATE OR REPLACE FUNCTION). L'étape [18/18] de sql_check.sh (qui EXÉCUTE le RPC en tant qu'app_user et vérifie total, attendu, ventilation par méthode, UPSERT sans duplication et le rejet des employés) tourne dans la CI (job sql-check, Postgres 16 jetable) — la syntaxe a déjà été validée localement (bash -n). Sans danger de le relancer autant de fois que nécessaire.
>
> Attention : Pas de fenêtre de compatibilité dure (contrairement à W11) : la Partie 25 ne fait qu'AJOUTER une table + un RPC — aucune donnée existante n'est réécrite. L'app doit être sur la version avec la clôture de caisse pour UTILISER le RPC, mais les clients hérités ne cassent pas en écrivant. Publier quand même la nouvelle version ensemble (étape 6) pour que la fonctionnalité apparaisse.

Étape 0 — Prérequis

- Accès au Dashboard du projet + mot de passe de la base

(Settings → Database → Connection string).

- L'hôte direct du projet est IPv6 uniquement → utiliser le pooler

aws-0-eu-north-1.pooler.supabase.com.

- psql local ou Docker (postgres:16 embarque psql).
- Attention : NE JAMAIS exécuter scripts/sql_check.sh contre la base live — il

fait DROP SCHEMA public CASCADE. La vérification live passe uniquement par scripts/sql_check_live.sh (lecture seule).

- App construite avec la fonctionnalité de clôture de caisse prête à

publier (voir l'étape 6).

Définir l'URL (remplacer SENHA) :

```
export
SUPABASE_DB_URL='postgresql://postgres.dotklbpqmlnhcdifiukd:SENHA@aws-0-eu-north-1.pooler.supabase.com:6543/
postgres?sslmode=require'
```

Étape 1 — Sauvegarde (obligatoire)

- Option rapide : Dashboard → Database → Backups → « Create a backup »

(ou activer le PITR).

- Option CLI (port 5432 *session* pooler, adapté à pg_dump) :

```
pg_dump
"postgresql://postgres.dotklbpqmlnhcdifiukd:SENHA@aws-0-eu-north-1.pooler.supabase.com:5432/postgres?sslmode=r
equire" \
 --schema=public -f backup_pre_cash_closing.sql
```

Étape 2 — Pré-vol (lecture seule, SQL Editor)

2.1 Partie 25 déjà appliquée ? Si la table cash_closings existe déjà, la migration a déjà eu lieu → passer à l'étape 4 (et vérifier l'étape 6) :

```
SELECT to_regclass('public.cash_closings') AS cash_closings_exists;
-- Attendu : NULL (non appliqué) ou public.cash_closings (déjà appliqué).
```

2.2 Clôtures existantes ? (bases qui peuvent avoir des lignes issues d'un test manuel du RPC avant ce runbook) :

```
SELECT count(*) AS existing_closings FROM public.cash_closings;
```

2.3 Échantillon du jour (pour vérifier plus tard que le calcul du caisse attendue côté serveur correspond aux ventes caisse d'aujourd'hui — noter le total des ventes caisse du jour). Attention : La requête utilise date_trunc('day', now()) dans le fuseau du SERVEUR (UTC sur Supabase), alors que le RPC reçoit le début du jour LOCAL du client converti en UTC — pour les magasins MZN (UTC+2) les deux « jours » peuvent diverger près de minuit. L'échantillon est informatif ; le RPC fait autorité :

```
SELECT COALESCE(SUM(total_amount), 0) AS cash_sales_today
FROM public.sales
WHERE status = 'active'
 AND payment_method = 'cash'
 AND date_time >= date_trunc('day', now())
 AND date_time < date_trunc('day', now()) + interval '1 day';
```

Étape 3 — Appliquer supabase/upgrade.sql

Option A — psql via le pooler, UNE transaction (recommandé — convention du projet)

```
psql "$SUPABASE_DB_URL" -X -v ON_ERROR_STOP=1 -1 -f supabase/upgrade.sql
```

- -1 = transaction unique : si une quelconque partie échoue, tout

est annulé — aucun état partiel.

- Attendu : aucune erreur (la Partie 25 n'émet que des NOTICE « already

exists » sur les colonnes/tables déjà créées — normal, c'est l'idempotence qui travaille).

Option B — SQL Editor du Dashboard

1. Ouvrir le SQL Editor du projet.
2. Coller le contenu complet de supabase/upgrade.sql.
3. Exécuter. L'éditeur exécute le script en un seul lot ; en cas d'erreur il

s'arrête et affiche le message. Comme le script est idempotent, corriger et relancer est sans danger.

Étape 4 — Vérification post-migration (lecture seule)

4.1 Exécuter le script de vérification live (lecture seule, maintenant avec 35 contrôles, ceux de la clôture de caisse étant 32–34 plus le contrôle d'aperçu 35) :

```
SUPABASE_DB_URL="$SUPABASE_DB_URL" bash scripts/sql_check_live.sh
```

- Attendu : VÉRIFICATION COMPLÈTE — tous les contrôles passés, y compris :
- 32 — table cash_closings + RLS ON + UNIQUE (store_id, closing_date) ;
- 33 — RPC close_cash_register SECURITY DEFINER + garde

is_manager_of_store + calcul côté serveur sur sales ;

- 34 — triggers user_id/store_id + index idx_cash_closings_store.

4.2 Contrôle manuel (SQL Editor) :

```
-- 1) Table + unicité (doit exister ; une clôture par magasin/jour) :
SELECT conname, pg_get_constraintdef(oid) AS def
FROM pg_constraint
WHERE conrelid = 'public.cash_closings'::regclass
 AND contype = 'u';
-- 2) RPC présent + SECURITY DEFINER + search_path fixé :
SELECT p.proname, p.prosecdef, p.proconfig
FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace
WHERE n.nspname = 'public' AND p.proname = 'close_cash_register';
-- 3) La migration n'a PAS créé de clôtures (elle n'insère aucune donnée) :
SELECT count(*) AS closings_after_migration FROM public.cash_closings;
-- Attendu : 0 (ou le comptage de l'étape 2.2, inchangé).
```

4.3 Comptages de lignes des tables métier — s'ils ont été notés à l'étape 2.3, vérifier que rien n'a changé (la Partie 25 ne touche pas aux données existantes).

Étape 5 — Idempotence (optionnelle, confirme que la réapplication ne duplique pas)

Répéter l'étape 3 et vérifier que la table a toujours un seul jeu d'objets :

```
-- Objets uniques (pas de doublons) :
SELECT count(*) AS closing_triggers FROM pg_trigger
WHERE tgrelid = 'public.cash_closings'::regclass;
-- Attendu : 2 (trg_cash_closings_user_id + trg_cash_closings_store_id).
SELECT count(*) AS closing_policies FROM pg_policies
WHERE schemaname = 'public' AND tablename = 'cash_closings';
-- Attendu : 4 (select/insert/update/delete).
```

Étape 6 — Publier l'app (rapport Z)

La Partie 25 ne réécrit pas de données, mais la fonctionnalité n'apparaît que dans la nouvelle UI :

1. Publier la version de l'app avec la clôture de caisse — bouton dans

Finance (propriétaire/gérant) → feuille du rapport Z avec le comptage physique de la caisse.

1. Les clients hérités continuent de fonctionner (le RPC est additif) ; la

fonctionnalité n'apparaît simplement pas tant qu'ils n'ont pas mis à jour.

1. La nouvelle app reste 100% compatible hors-ligne (le SQLite local crée

déjà la table cash_closings à l'ouverture de la base).

Rollback (seulement si nécessaire)

> La Partie 25 est additive (table + RPC — aucune donnée existante n'est réécrite). Pour revenir en arrière, il suffit de ne pas publier la nouvelle app (la fonctionnalité n'apparaît pas) et, si vous voulez supprimer la table, de restaurer depuis la sauvegarde de l'étape 1. Il n'y a aucune migration destructive à défaire.

Checklist finale

#	Élément	Vérification
1	Sauvegarde effectuée	backup_pre_cash_closing.sql existe
2	Pré-vol propre	cash_closings n'existe pas ; échantillon noté
3	upgrade.sql appliqué	aucune erreur (transaction unique)
4	sql_check_live.sh	tous les 35 contrôles (clôture 32–34 + aperçu serveur 35)
5	Unicité par magasin/jour	UNIQUE (store_id, closing_date) présent
6	Réapplication sans effet	idempotence confirmée (étape 5)
7	App publiée	version avec clôture de caisse publiée

8	La clôture fonctionne	propriétaire/gérant clôturent le tiroir ; employé bloqué (serveur)